INF224 Proje Ödevi
TinySQL – Mini Veritabanı Motoru Teknik Raporu

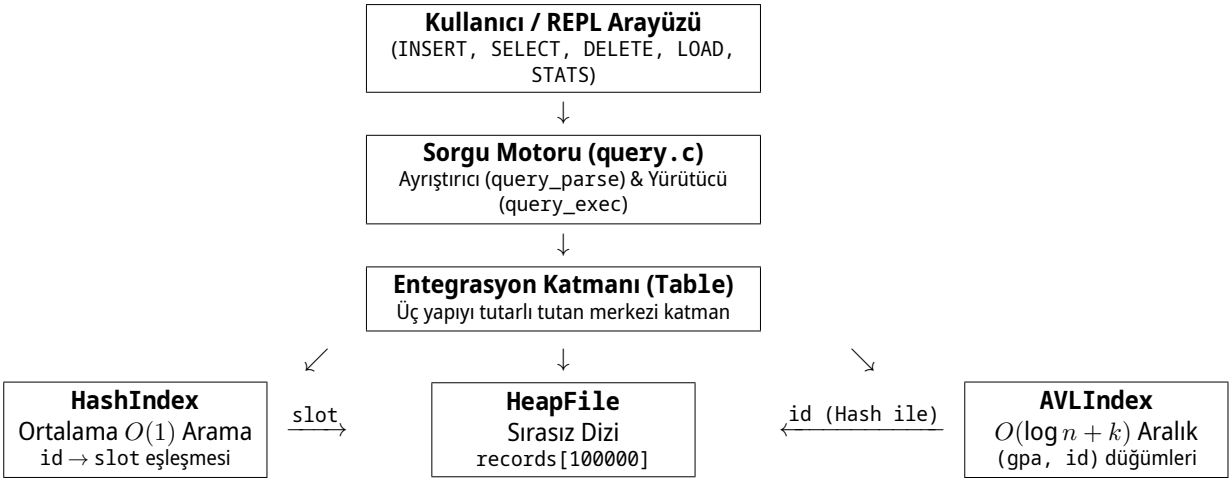
Muhammed Enis Kartal (21401921) Yusuf Ziya Demirbaş (21401924)

4 Kasım 2026

1 Sistem Mimarisi (Bölüm 13)

TinySQL veritabanı motoru, veri tutarlılığını ve modülerliği sağlamak amacıyla katmanlı bir mimariyle tasarlanmıştır. Sistemin merkezinde, tüm fiziksel kayıtları bellekte ardışık olarak barındıran `HeapFile` yer alır. Verilere hızlı erişim sağlamak için birincil anahtar (`id`) üzerinde `HashIndex` ve not ortalaması (gpa) üzerinde `AVLIndex` yapıları konumlandırılmıştır.

Dış dünyadan (kullanıcı etkileşimli REPL kabuğu veya test araçları üzerinden) gelen tüm ekleme, silme ve sorgulama işlemleri yalnızca entegrasyon katmanı olan `Table` modülü üzerinden yürütülür. `Table` dışındaki hiçbir modül `HeapFile`, `HashIndex` veya `AVLIndex` yapılarına doğrudan yazma işlemi yapmaz; bu sayede üç veri yapısı arasındaki senkronizasyon garanti altına alınır.



Bileşenlerin Görevleri ve İlişkileri:

- **`HeapFile` (`heap.c / heap.h`):** Öğrenci kayıtlarının (`Record`) fiziksel olarak tutulduğu `MAX_RECORDS` (100.000) kapasiteli sabit boyutlu dizidir. `Table` yapısı yaklaşık 10 MB büyüklüğünde olduğu için yığın (stack) taşmasını önlemek adına `table_create()` ile dinamik bellekte (`malloc`) ayrılır.
- **`HashIndex` (`hash_index.c / hash_index.h`):** Birincil anahtar olan `id` değerini, kaydın `HeapFile` içindeki fiziksel dizi indeksine (`slot`) dönüştürür. Fiziksel kaydın kopyasını tutmaz; yalnızca 4 baytlık `id` ve `slot` bilgisini saklar.
- **`AVLIndex` (`avl_index.c / avl_index.h`):** Öğrencilerin gpa değerlerini sıralı tutan dengeli ikili arama ağacıdır. Her düğümde yalnızca sıralama anahtarı gpa ve uydu veri olarak `id` tutulur. Aralık sorgusunda bulunan `id` değerleri, `HashIndex` üzerinden $O(1)$ sürede fiziksel kayda çevrilir.

2 Veri Yapısı Kararları (Bölüm 14)

Projenin hem teorik zaman karmaşıklığı hedeflerini sağlaması hem de bellek güvenliği açısından kararlı çalışması için aşağıdaki tasarım kararları alınmıştır:

2.1 Heap Dosyasında "Swap-with-Last" ve $O(1)$ Silme Senkronizasyonu

Dizi tabanlı bir depolama katmanında ortadaki bir eleman silindiğinde oluşan boşluğu kapatmak için sağdaki tüm elemanları birer birim sola kaydırmak $O(n)$ maliyet doğurur. Bunun yerine, dizideki son kayıt silinen kaydın bulunduğu indekse (`slot`) taşınmış (*swap-with-last*) ve dizi boyutu (`count`) bir azaltılmıştır.

Ancak, tek başına `heap_delete(hf, id)` çağrıldığında silinecek elemanın yerini bulmak için doğrusal tarama ($O(n)$) yapılması gerekir. `Table` katmanı silinecek kaydın fiziksel `slot` numarasını `hash_lookup` ile zaten ortalama $O(1)$ sürede bildiği için, doğrudan bilinen slottaki kaydı $O(1)$ sürede silen `heap_delete_slot(hf, slot)` mekanizması kullanılmıştır.
Silme sonucunda son slottan boşalan slota taşınan kaydın `HashIndex` içindeki konum bilgisi, yeni bellek tahsisi (`free`/`malloc`) yapılmadan `hash_update(hi, moved_id, freed_slot)` fonksiyonuyla yerinde (in-place) ortalama $O(1)$ sürede güncellenmiştir. Böylece `table_delete` işleminin toplam karmaşıklığı gerçek anlamda $O(\log n)$ seviyesinde tutulmuştur.

2.2 Hash Tablosunda Ayrı Zincirleme (Separate Chaining) ve Tablo Boyutu

Çakışmaları (collision) çözmek için *Açık Adresleme (Open Addressing)* yerine *Ayrı Zincirleme (Separate Chaining)* tercih edilmiştir:

- **Silme Kararlılığı:** Veritabanında sık DELETE işlemi yapıldığında açık adresleme yöntemleri silinen hücrelere *mezar taşı (tombstone)* bırakmak zorundadır. Zamanla biriken bu işaretler arama zincirlerini uzatarak performansı $O(1)$'den uzaklaştırır. Ayrı zincirlemede ise silinen düğüm bağlı listeden çıkarılıp doğrudan `free()` ile bellekten temizlenir.
- **Kova Sayısı Seçimi (`TABLE_SIZE = 20011`):** Ödev föyünde başlangıç önerisi olarak verilen 10.007 kova sayısı, 10.000 kayıtta yük faktörünü $\alpha \approx 1.0$ seviyesine çıkarmaktadır. Yük faktörünü $\alpha = 0.50$ seviyesinde tutarak zincir uzunluklarını kısaltmak adına tablo boyutu **20.011** (asal sayı) olarak belirlenmiştir.
- **Hash Fonksiyonu ve Birikimli Çakışma Sayacı:** Ardışık veya belirli örüntüdeki `id` değerlerinin aynı kovalarda kümelenmesini önlemek için bit kaydırma (`XOR-shift`) ve `0x45d9f3b` sabitiyle çarpma yapan karıştırıcı fonksiyon kullanılmıştır. Dolu bir kovaya yapılan her ekleme 1 çakışma olarak sayılır ve istatistiksel geçmişi korumak adına silme işlemlerinde çakışma sayacı azaltılmaz.

2.3 AVL Ağacında Bileşik Anahtar `(gpa, id)` ve Sınır Kontrolleri

Not ortalaması (gpa) benzersiz bir alan değildir; binlerce öğrenci aynı GPA değerine sahip olabilir. Yinelenen anahtarları yönetmek için düğüm içinde bağlı liste tutmak yerine **`(gpa, id)` bileşik anahtarı (composite key)** tercih edilmiştir:

- **Karşılaştırma Kuralı:** Önce gpa değerleri karşılaştırılır; eşitlik durumunda birincil anahtar olan `id` değerine bakılır. `id` benzersiz olduğu için ağaçtaki her düğüm mutlak suretle benzersiz hale gelir.
- **Neden Bağlı Liste Yerine Bileşik Anahtar?** Aynı GPA değerine sahip çok sayıda kayıt eklendiğinde bile ağaç dengeli kalır ve `avl_delete(root, gpa, id)` çağrısı ikinci bir liste taramasına gerek duymadan doğrudan tek bir düğümü $O(\log n)$ sürede siler.
- **Aralık Sorgusu (`avl_range`) Sınır Eşitliği:** Bileşik anahtar yapısında, bir düğümle aynı GPA değerine sahip başka düğümler (farklı `id` değerleri nedeniyle) o düğümün hem sol hem de sağ alt ağacında yer alabilir. Bu nedenle `avl_range` fonksiyonu sınır değerlerine eşit düğümlerde de (`root->key >= lo` ve `root->key <= hi`) alt ağaçlara inmeye devam ederek sınır değerindeki hiçbir kaydın kaçırılmamasını sağlar.
- **Kapsamlı Doğrulama (`avl_verify`):** Fonksiyon yalnızca yükseklik farkını değil; her düğümde `|sol - sağ| <= 1` denge şartını, `height` alanının alt ağaçlarla tutarlılığını ve *in-order* gezinme sırasının `(gpa, id)` çiftine göre kesin artan olup olmadığını denetler.

3 Performans Tablosu ve Ölçüm Yöntemi (Bölüm 15)

3.1 Ölçüm Yöntemi (`tools/bench.c` ve `tools/gen_csv.c`)

Tekil bir Hash veya AVL işleminin süresi (nanosaniye mertebesi), standart C kütüphanesindeki `clock()` fonksiyonunun çözünürlüğünün ($\approx 1\ \mu$s) çok altındadır. Doğru ve tekrarlanabilir ölçümler elde etmek için aşağıdaki metodoloji uygulanmıştır:

1. **Tekrarlı Ölçüm Döngüsü:** Her ölçüm, toplam geçen süre en az **0.1 saniye** olana kadar döngü içinde tekrarlanmış ve toplam süre işlem sayısına bölünerek işlem başına ortalama süre hesaplanmıştır. Her turda 1.000 işlem koşturularak `clock()` çağrısının kendi ek yükü (overhead) etkisizleştirilmiştir.
2. **Derleyici Optimizasyonu Koruması (`volatile`):** `-O2` optimizasyon seviyesinde derleyicinin sonucu kullanılmayan arama döngülerini tamamen silmesini (*dead-code elimination*) engellemek için arama sonuçları `volatile` bir değişkende toplanmıştır.
3. **Platform Bağımsız Veri Üretimi:** Standart `rand()` fonksiyonu farklı işletim sistemlerinde farklı diziler ürettiği için `tools/gen_csv.c` içinde sabit tohumlu (tohum = 42) Doğrusal Eşlikli Üreteç (LCG) kullanılmıştır. Böylece hem macOS hem de Linux ortamlarında birebir aynı 10.000 kayıtlık veri kümesi üzerinde ölçüm yapılmıştır.

3.2 Nokta Sorgusu: Doğrusal Arama vs. Hash İndeksi (Madde 1–5)

Tabloda mevcut olan 1.000 rastgele `id` için yapılan aramaların işlem başına ortalama süreleri Tablo 1'de sunulmuştur:

Kayıt Sayısı (n)	**Heap Araması $O(n)$ (ns)**	**Hash Araması $O(1)$ (ns)**	**Hızlanma**
100	23.9	2.0	12×
1.000	130.2	2.6	50×
10.000	1432.9	3.3	434×

Tablo 1: Nokta sorgusu süre karşılaştırması (Test senaryosu, madde 1–5)

3.3 Yük Faktörünün (α) Aramaya Etkisi (Madde 6)

Kova sayısı sabit (`TABLE_SIZE = 20011`) tutularak kayıt sayısı $n = 5.000$'den 80.000'e kadar artırılmış ve yük faktörünün ($\alpha = n/m$) arama performansına etkisi incelenmiştir (Tablo 2).

n	α	**Dolu Kova Oranı**	**En Uzun Zincir**	**Ort. Karşılaştırma (Ölçülen)**	**Ort. Karşılaştırma (Teori: $1 + \alpha/2$)**	**Hash Araması (ns)**
5.000	0.25	0.22	4	1.122	1.125	3.1
10.000	**0.50**	**0.39**	**5**	**1.258**	**1.250**	**3.6**
14.000	0.70	0.50	6	1.350	1.350	3.4
20.000	1.00	0.64	7	1.492	1.500	3.6
40.000	2.00	0.87	9	1.995	1.999	4.5
80.000	4.00	0.98	15	2.994	2.999	6.1

Tablo 2: Yük faktörünün (α) zincir uzunluğu, karşılaştırma sayısı ve arama süresine etkisi (Madde 6)

Yük Faktörü Yorumu: Ayrı zincirlemeli bir hash tablosunda başarılı bir aramada beklenen teorik karşılaştırma sayısı $1 + \frac{\alpha}{2}$ formülüyle hesaplanır. Tablo 2'de görüldüğü üzere, deneysel olarak ölçülen ortalama karşılaştırma sayıları teorik değerlerle virgülden sonra iki basamak hassasiyetle örtüşmektedir (örneğin $n = 10.000, \alpha = 0.50$ için teorik 1.250, ölçülen 1.258). α değeri 0.50'den 4.00'a (8 katına)

çıktığında en uzun zincir 5'ten 15'e yükselmekte ve ortalama arama süresi 3.6 ns'den 6.1 ns'ye çıkmaktadır. Bu sonuç, kova sayısının 20.011 seçilerek $\alpha = 0.50$ tutulmasının çakışmaları sınırlandırdığını ve sabit zamanlı erişimi koruduğunu kanıtlamaktadır.

3.4 AVL Ağacı Doğrulaması ve Aralık Sorgusu (Madde 7, 8 ve 9)

Farklı kayıt sayılarında AVL ağacının yüksekliği, teorik üst sınır olan $\lfloor 1.44 \log_2 n \rfloor$ ile karşılaştırılmış ve `avl_verify` ile yapısal doğrulaması yapılmıştır (Tablo 3). Ardından GPA $\in [3.0, 3.5]$ aralık sorgusu doğrusal tarama ile kıyaslanmıştır (Tablo 4).

n	**AVL Yüksekliği**	**Teorik Üst Sınır** $\lfloor 1.44 \log_2 n \rfloor$	**`avl_verify` Hata Sayısı**	**Sınır İçinde**
100	8	9	0	Evet
1.000	12	14	0	Evet
10.000	16	19	0	Evet

Tablo 3: AVL yüksekliği ve doğrulama sonuçları (Madde 7 ve 9)

n	k **(AVL)**	k **(Doğrusal)**	**AVL** $O(\log n + k)$ **(**μ**s)**	**Doğrusal Tarama** $O(n)$ **(**μ**s)**	**Hızlanma**
100	16	16	0.045	0.065	1.4×
1.000	137	137	0.301	0.569	1.9×
10.000	1215	1215	3.292	19.561	5.9×

Tablo 4: GPA $\in [3.0, 3.5]$ aralık sorgusu karşılaştırması (Madde 8)

3.5 REPL Gösterimi ve Komut Arayüzü Kanıtı (Madde 10–11)

`make demo` komutu ile `data/demo.sql` betiği çalıştırılmış; 10.000 kayıt yüklendikten sonra beş farklı sorgu türü ardışık olarak yürütülmüştür (`report/repl_cikti.txt`):

```
TinySQL REPL Başlatıldı.
tinysql> LOAD data/students.csv
10000 kayıt yüklendi.
Süre: 0.004977 s
tinysql> STATS
=== TinySQL İstatistikleri ===
Toplam kayıt       : 10000 / 100000
Hash yük faktörü   : 0.50  |  Dolu kova oranı: 0.39  |  Çakışmalar: 2110
AVL yüksekliği     : 16  |  Teorik üst sınır: 19
AVL doğrulama      : 0 hata
Son sorgu süresi   : 0.004977 s
tinysql> INSERT 100500 "Aylin Demir" "Felsefe" 3.72 2
Kayıt eklendi.
Süre: 0.000001 s
tinysql> SELECT id = 100500
1 kayıt bulundu.
  100500 | Aylin Demir | Felsefe | 3.72 | 2
Süre: 0.000000 s
tinysql> SELECT gpa BETWEEN 3.0 3.5
1274 kayıt bulundu.
  1445 | Elif Yıldız | Fizik | 3.00 | 4
  ... ve 1264 kayıt daha
Süre: 0.000010 s
tinysql> SELECT dept = "Bilgisayar"
1939 kayıt bulundu.
  20108 | Mustafa Yılmaz | Bilgisayar | 3.53 | 2
```

```
  ... ve 1929 kayıt daha
Süre: 0.000065 s
tinysql> SELECT dept = "Felsefe"
1 kayıt bulundu.
  100500 | Aylin Demir | Felsefe | 3.72 | 2
Süre: 0.000028 s
tinysql> DELETE 100500
Kayıt silindi.
Süre: 0.000007 s
tinysql> SELECT id = 100500
Kayıt bulunamadı.
Süre: 0.000000 s
tinysql> EXIT
```

3.6 Aynı Sonucu Veren SELECT id ve SELECT dept Karşılaştırması (Madde 12)

Tabloya bölümü "Felsefe" olan tek bir kayıt (`id = 100500`) eklenmiş ve aynı tekil kaydı döndüren indeksli `SELECT id = 100500` sorgusu ile indekssiz `SELECT dept = "Felsefe"` sorgusu karşılaştırılmıştır (Tablo 5).

n	SELECT id Sonucu	SELECT dept Sonucu	Hash id (ns)	Tarama dept (ns)	Fark
101	id 100500	1 kayıt (id 100500)	2.4	193.2	79×
1.001	id 100500	1 kayıt (id 100500)	2.5	1834.1	728×
10.001	id 100500	1 kayıt (id 100500)	2.4	17934.2	7363×

Tablo 5: Aynı sonucu veren indeksli (`id`) ve indekssiz (`dept`) sorguların karşılaştırması (Madde 12)

4 Big-O Analizi: Teorik ve Deneysel Karşılaştırma (Bölüm 16)

İşlem	HeapFile	HashIndex (Ort.)	AVLIndex	Entegre Table
Ekleme (INSERT)	$O(1)$	$O(1)$	$O(\log n)$	$O(\log n)$
Silme (DELETE)	$O(1)$ (slot ile)	$O(1)$	$O(\log n)$	$O(\log n)$
Nokta Sorgusu (SELECT id)	$O(n)$	$O(1)$	–	$O(1)$
Aralık Sorgusu (SELECT gpa)	$O(n)$	–	$O(\log n + k)$	$O(\log n + k)$
Bölüm Sorgusu (SELECT dept)	$O(n)$	–	–	$O(n)$

Tablo 6: Modül bazında teorik zaman karmaşıklığı özeti

- **Nokta Sorgusu ($O(1)$ vs $O(n)$):** Kayıt sayısı $n = 100$'den 10.000'e (100 katına) çıktığında doğrusal tarama süresi 23.9 ns'den 1432.9 ns'ye (≈ 60 kat) ve SELECT dept string karşılaştırma süresi 193.2 ns'den 17934.2 ns'ye (≈ 93 kat) artmıştır. Buna karşın Hash araması 2.0–3.3 ns aralığında sabit kalarak $O(1)$ davranışını deneysel olarak doğrulamıştır.
- **Aralık Sorgusu ($O(\log n + k)$ vs $O(n)$):** GPA $\in [3.0, 3.5]$ aralığı tüm kayıtların yaklaşık %12'sini ($k = 1215$) döndürmektedir. Alt ağaç budama (*pruning*) sayesinde AVL ağacı aralık dışı dalları ziyaret etmemiş ve $n = 10.000$'de doğrusal taramaya kıyasla $5.9\times$ hızlanma sağlamıştır. Daha dar aralıklarda ($k \ll n$) hızlanma oranı logaritmik erişim sayesinde çok daha yüksek olmaktadır.
- **Ekleme ve Silme ($O(\log n)$):** `HeapFile` sona ekleme ($O(1)$) ve bilinen slottan *swap-with-last* ile silme ($O(1)$) yaptığından, `HashIndex` ise ekleme, silme ve slot güncellemesini (`hash_update`) ortalama $O(1)$ sürede tamamladığından, `Table` üzerindeki ekleme ve silme maliyetini belirleyen baskın terim AVL ağacının $O(\log n)$ yeniden dengeleme süresidir.

5 Grup Katkı Dağılımı (Bölüm 17)

Proje geliştirme süreci, önce modül sözleşmelerinin (`.h` başlık dosyaları ve `README.md`) belirlenmesi, ardından çekirdek veri yapılarının kodlanması ve son aşamada doğrulama/ölçüm altyapısının kurulması şeklinde yürütülmüştür:

- **Muhammed Enis Kartal (21401921):**
  - `src/heap.c`, `src/hash_index.c`, `src/avl_index.c`, `src/table.c` ve `src/query.c` modüllerinin ilk sürümlerinin kodlanması ve `heap_delete_slot` ile $O(\log n)$ silme entegrasyonu.
  - Tüm kaynak dosyalardaki fonksiyonların üzerine ödev formatına uygun Big-O karmaşıklık yorumlarının eklenmesi.
  - Linux (`WSL / Kali`) ortamında derleme, `valgrind` bellek testleri ve teknik raporun LaTeX formatında hazırlanması (Bölüm 13 ve 14'ün yazımı, tüm bölümlerin dizgisi).
- **Yusuf Ziya Demirbaş (21401924):**
  - Proje iskeletinin, modül sözleşmelerinin (6 adet `.h` başlık dosyası, `Makefile`, `README.md`) ve mimari kuralların oluşturulması.
  - `avl_range` içinde eşit GPA değerine sahip kayıtların atlanmasını önleyen sınır düzeltmesinin yapılması ve `avl_verify` fonksiyonunun sıralama/yükseklik denetimiyle genişletilmesi.
  - Sorgu motorunda ödevdeki komut biçiminin (`SELECT id = ...`), kayıt/süre çıktılarının, `STATS` formatının ve `sscanf` taşma sınırlarının (`%63`, `%31`) uygulanması.
  - Platform bağımsız CSV üreticisinin (`tools/gen_csv.c`), otomatik ölçüm aracının (`tools/bench.c` – Tablo 1–5) ve REPL demo betiğinin (`make demo`) geliştirilmesi; macOS `leaks` ve Linux `valgrind` (0 sızıntı) bellek kontrollerinin yapılması.